



北京邮电大学

BEIHANG UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第五章 卷积码

李丽香, 彭海朋

北京邮电大学网络空间安全学院

网络与交换技术国家重点实验室





北京邮电大学

卷积码的发展历史

1955年, 埃利斯(P. Elias)首次提出卷积码的概念

1957年, J. M. Wozencraft提出卷积码的序列译码算法

1963年, J. L. Massey提出较易实现的门限译码算法

1967年, A. J. Viterbi提出卷积码的一种最大似然译码算法

卷积码在深空通信、卫星通信、移动通讯等领域得到了应用

现实中两个真理之间的最短路径要经过复杂的中间过程。—— Jacques Hadamard (1865—1963)

Page 122



北京邮电大学

卷积码与分组码的区别

- ❑ 卷积码有记忆性, 分组码无记忆性
- ❑ 卷积码充分利用了各组信息之间的相关性, 信息序列不被分段, 而是被连续处理
- ❑ 通常, 卷积码的n和k比分组码的n和k要小的多
- ❑ 在同样的编码效率下, 卷积码的性能优于分组码
- ❑ 在相似的纠错能力下, 卷积码的实现比分组码简单
- ❑ 卷积码必须用**序列逻辑电路**来实现, 更适用于前向纠错系统; 分组码是用**组合逻辑电路**来实现

现实中两个真理之间的最短路径要经过复杂的中间过程。—— Jacques Hadamard (1865—1963)

Page 123



北京邮电大学

离散序列的非循环卷积运算

定义: 设f(n)和g(n)是两个序列, 则序列f和g的离散卷积运算为

$$(f * g)(m) = \sum_n f(n)g(m-n)$$

举例: 若f=(10011), g=(111), 则f*g的计算过程如下

$\begin{array}{r} (f * g)(0) = \\ \begin{array}{r} \times \\ 11001 \\ + \\ 111 \\ \hline \end{array} \end{array}$	= 1	$\begin{array}{r} (f * g)(4) = \\ \begin{array}{r} \times \\ 11001 \\ + \\ 111 \\ \hline \end{array} \end{array}$	= 0
$\begin{array}{r} (f * g)(1) = \\ \begin{array}{r} \times \\ 11001 \\ + \\ 111 \\ \hline \end{array} \end{array}$	= 1	$\begin{array}{r} (f * g)(5) = \\ \begin{array}{r} \times \\ 11001 \\ + \\ 111 \\ \hline \end{array} \end{array}$	= 0
$\begin{array}{r} (f * g)(2) = \\ \begin{array}{r} \times \\ 11001 \\ + \\ 111 \\ \hline \end{array} \end{array}$	= 1	$\begin{array}{r} (f * g)(6) = \\ \begin{array}{r} \times \\ 11001 \\ + \\ 111 \\ \hline \end{array} \end{array}$	= 1
$\begin{array}{r} (f * g)(3) = \\ \begin{array}{r} \times \\ 11001 \\ + \\ 111 \\ \hline \end{array} \end{array}$	= 1		

故 f*g=(11111001)

Page 124



北京邮电大学

二元(2,1,2)卷积码的编码器

编码器主要由m=2级移位寄存器, n=2个模2加法器组成。所有的卷积码编码器都可以用这种类型的线性前馈移位寄存器来实现。

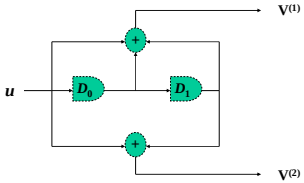


图6.1 二元(2,1,2)卷积码的编码器的电路图

Page 125



北京邮电大学

(2,1,2)卷积码的冲激响应

冲激响应: 编码器的冲激响应就是通过令u=(100...)所得到的两个输出序列。

因为编码器有m个存储单元, 所以冲激响应(生成序列)至多持续m+1个时间单元, 且可以写成

$$g^{(i)} = (g_0^{(i)}, g_1^{(i)}, \dots, g_m^{(i)}), g^{(2)} = (g_0^{(2)}, g_1^{(2)}, \dots, g_m^{(2)})$$

其中上标表示第几个输出端

举例: 图6.1的(2,1,2)卷积码的冲激响应为g⁽¹⁾=(111), g⁽²⁾=(101)

另外, 冲激响应向量还可以表示输入、移位寄存器和输出之间的连接关系, 1表示有连接, 0表示没有连接

Page 126

信息序列 $u=(u_1, u_2, u_3, \dots)$ 每次进入编码器1比特，编码器的两个输出序列

$$v^{(1)} = (v_0^{(1)}, v_1^{(1)}, v_2^{(1)}, \dots), v^{(2)} = (v_0^{(2)}, v_1^{(2)}, v_2^{(2)}, \dots)$$

可以通过输入序列 u 和编码器的两个冲激响应作卷积运算而得到。

举例：若 $u=(1011)$ ，图6.1的(2,1,2)卷积码的冲激响应为 $g^{(1)}=(111)$ ， $g^{(2)}=(101)$ ，则该卷积码的输出序列为 $V^{(1)}=u*g^{(1)}=(110001)$ ， $V^{(2)}=u*g^{(2)}=(100111)$

Page 127

输出序列分别为 $v^{(1)} = (v_0^{(1)}, v_1^{(1)}, v_2^{(1)}, \dots)$ ， $v^{(2)} = (v_0^{(2)}, v_1^{(2)}, v_2^{(2)}, \dots)$

的(2,1,m)的卷积码的码字，为 $V^{(1)}$ 和 $V^{(2)}$ 的交错，即

$$v = (v_0^{(1)}, v_0^{(2)}, v_1^{(1)}, v_1^{(2)}, v_2^{(1)}, v_2^{(2)}, \dots)$$

举例：图6.1的(2,1,2)卷积码，若 $u=(1011)$ ，冲激响应为 $g^{(1)}=(111)$ ， $g^{(2)}=(101)$ ，该卷积码的输出序列为 $V^{(1)}=u*g^{(1)}=(110001)$ ， $V^{(2)}=u*g^{(2)}=(100111)$ ，得到的码字为 $v=(11,10,00,01,01,11)$

Page 128

以 $u=(u_1, u_2, u_3, \dots)$ 为输入，以

$$g^{(1)} = (g_0^{(1)}, g_1^{(1)}, \dots, g_n^{(1)}), g^{(2)} = (g_0^{(2)}, g_1^{(2)}, \dots, g_n^{(2)})$$

为生成序列的(2,1,m)卷积码的输出序列分别由方程 $V^{(1)}=u*g^{(1)}$ 和 $V^{(2)}=u*g^{(2)}$ 得到。

举例：图6.1的(2,1,2)卷积码的编码方程为

$$\begin{cases} V_i^{(1)} = u_i + u_{i-1} + u_{i-2} \\ V_i^{(2)} = u_i + u_{i-2} \end{cases} \quad [V_i^{(1)} \ V_i^{(2)}] = [u_i \ u_{i-1} \ u_{i-2}] \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

另外，编码方程中每一项的系数，还可以表示输入、移位寄存器和输出之间的连接关系，1表示有连接，0表示没有连接。

Page 129

(2,1,m)卷积码的生成矩阵：将生成序列 $g^{(1)}$ 和 $g^{(2)}$ 交织后形成的半无限矩阵

$$G = \begin{bmatrix} g_0^{(1)} & g_0^{(2)} & g_1^{(1)} & g_1^{(2)} & g_2^{(1)} & g_2^{(2)} & \dots & g_n^{(1)} & g_n^{(2)} & 0 & 0 & 0 & 0 & \dots \\ 0 & 0 & g_0^{(1)} & g_0^{(2)} & g_1^{(1)} & g_1^{(2)} & \dots & g_{n-1}^{(1)} & g_{n-1}^{(2)} & g_n^{(1)} & g_n^{(2)} & 0 & 0 & \dots \\ 0 & 0 & 0 & 0 & g_0^{(1)} & g_0^{(2)} & \dots & g_{n-2}^{(1)} & g_{n-2}^{(2)} & g_{n-1}^{(1)} & g_{n-1}^{(2)} & g_n^{(1)} & g_n^{(2)} & \dots \end{bmatrix}$$

举例：图6.1的(2,1,2)卷积码冲激响应为 $g^{(1)}=(111)$ ， $g^{(2)}=(101)$ ，该码的生成矩阵为

$$G = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 1 & \dots \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & \dots \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 & \dots \end{bmatrix}$$

Page 130

以 $u=(u_1, u_2, u_3, \dots)$ 为输入， G 为生成矩阵的(2,1,m)的卷积码的码字为 $V=uG$ 。

举例：图6.1的(2,1,2)卷积码，若 $u=(1011)$ ，则码字为 $v=(11,10,00,01,01,11)$

$$\begin{aligned} V &= uG \\ &= [1011] \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 1 & & & \\ & 1 & 1 & 1 & 0 & 1 & 1 & & \\ & & 1 & 1 & 0 & 1 & 1 & & \\ & & & 1 & 1 & 1 & 0 & 1 & 1 \end{bmatrix} \\ &= (11,10,00,01,01,11) \end{aligned}$$

Page 131

编码器主要有2个输入，3个输出，主要由 $m=1$ 级移位寄存器， $n=3$ 个模2加法器组成。

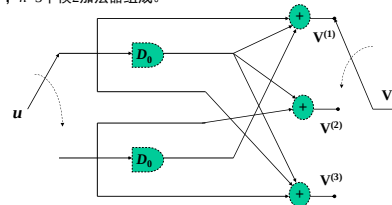


图6.2 二元(3,2,1)卷积码的编码器的电路图

Page 132



二元(3,2,m)卷积码的输入序列

对于一般的(3,2,m)卷积码, 由于k=2, 即每次有2比特进入编码器, 所以输入序列u可以写成

$$u = (u_0^{(1)}, u_0^{(2)}, u_1^{(1)}, u_1^{(2)}, u_2^{(1)}, u_2^{(2)}, \dots)$$

或分开写成2个序列

$$\begin{aligned} u^{(1)} &= (u_0^{(1)}, u_1^{(1)}, u_2^{(1)}, \dots) \\ u^{(2)} &= (u_0^{(2)}, u_1^{(2)}, u_2^{(2)}, \dots) \end{aligned}$$

Page 133



二元(3,2,m)卷积码的冲激响应

令 $u^{(1)}=(100\dots)$, 得到3个输出序列(长度均为 $m+1$)

$$\begin{aligned} g_1^{(1)} &= (g_{10}^{(1)}, g_{11}^{(1)}, g_{12}^{(1)}, \dots, g_{1m}^{(1)}) \\ g_2^{(1)} &= (g_{20}^{(1)}, g_{21}^{(1)}, g_{22}^{(1)}, \dots, g_{2m}^{(1)}) \\ g_3^{(1)} &= (g_{30}^{(1)}, g_{31}^{(1)}, g_{32}^{(1)}, \dots, g_{3m}^{(1)}) \end{aligned}$$

再令 $u^{(2)}=(100\dots)$, 又得到3个输出序列

$$\begin{aligned} g_1^{(2)} &= (g_{10}^{(2)}, g_{11}^{(2)}, g_{12}^{(2)}, \dots, g_{1m}^{(2)}) \\ g_2^{(2)} &= (g_{20}^{(2)}, g_{21}^{(2)}, g_{22}^{(2)}, \dots, g_{2m}^{(2)}) \\ g_3^{(2)} &= (g_{30}^{(2)}, g_{31}^{(2)}, g_{32}^{(2)}, \dots, g_{3m}^{(2)}) \end{aligned}$$

则称下面的序列为(3, 2, m)卷积码的生成序列

$$g_i^{(j)} = (g_{i0}^{(j)}, g_{i1}^{(j)}, g_{i2}^{(j)}, \dots, g_{im}^{(j)}), i=1,2, j=1,2,3$$

Page 134



二元(3, 2, 1)卷积码的冲激响应

举例: 图6.2的(3, 2, 1)卷积码的生成序列为

$$\begin{aligned} g_1^{(1)} &= (g_{10}^{(1)}, g_{11}^{(1)}, g_{12}^{(1)}, \dots, g_{1m}^{(1)}) = (11) \\ g_2^{(1)} &= (g_{20}^{(1)}, g_{21}^{(1)}, g_{22}^{(1)}, \dots, g_{2m}^{(1)}) = (01) \\ g_3^{(1)} &= (g_{30}^{(1)}, g_{31}^{(1)}, g_{32}^{(1)}, \dots, g_{3m}^{(1)}) = (11) \end{aligned}$$

$$\begin{aligned} g_1^{(2)} &= (g_{10}^{(2)}, g_{11}^{(2)}, g_{12}^{(2)}, \dots, g_{1m}^{(2)}) = (01) \\ g_2^{(2)} &= (g_{20}^{(2)}, g_{21}^{(2)}, g_{22}^{(2)}, \dots, g_{2m}^{(2)}) = (10) \\ g_3^{(2)} &= (g_{30}^{(2)}, g_{31}^{(2)}, g_{32}^{(2)}, \dots, g_{3m}^{(2)}) = (10) \end{aligned}$$

Page 135



(3, 2, 1)卷积码的码字

输出序列分别为

$$v^{(1)} = (v_0^{(1)}, v_1^{(1)}, v_2^{(1)}, \dots), v^{(2)} = (v_0^{(2)}, v_1^{(2)}, v_2^{(2)}, \dots), v^{(3)} = (v_0^{(3)}, v_1^{(3)}, v_2^{(3)}, \dots)$$

的(3,2,m)的卷积码的码字为

$$v = (v_0^{(1)}, v_0^{(2)}, v_0^{(3)}, v_1^{(1)}, v_1^{(2)}, v_1^{(3)}, v_2^{(1)}, v_2^{(2)}, v_2^{(3)}, \dots)$$

举例: 图6.2的(3, 2, 1)卷积码, 若 $u^{(1)}=(101)$, $u^{(2)}=(110)$

$$V^{(1)} = u^{(1)} * g_1^{(1)} + u^{(2)} * g_2^{(1)} = (101) * (11) + (110) * (01) = (1111) + (0110) = (1001)$$

$$V^{(2)} = u^{(1)} * g_1^{(2)} + u^{(2)} * g_2^{(2)} = (101) * (01) + (110) * (10) = (0101) + (1100) = (1001)$$

$$V^{(3)} = u^{(1)} * g_1^{(3)} + u^{(2)} * g_2^{(3)} = (101) * (11) + (110) * (10) = (1111) + (1100) = (0011)$$

得到的码字为 $v=(110, 000, 001, 111)$ 。

Page 136



二元(3, 2, 1)卷积码的编码方程

举例: 图6.2的(3,2,1)卷积码的编码方程为

$$\begin{cases} V_i^{(1)} = u_i^{(1)} + u_{i-1}^{(1)} + u_{i-1}^{(2)} \\ V_i^{(2)} = u_i^{(2)} + u_{i-1}^{(1)} \\ V_i^{(3)} = u_i^{(1)} + u_i^{(2)} + u_{i-1}^{(1)} \end{cases}$$

另外, 编码方程中每一项的系数, 还可以表示输入、移位寄存器和输出之间的连接关系, 1表示有连接, 0表示没有连接

Page 137



(3, 2, m)卷积码的生成矩阵

(3, 2, m)卷积码的生成序列为 $g_i^{(j)}, i=1,2, j=1,2,3$

(3, 2, m)卷积码的生成矩阵为

$$G = \begin{bmatrix} g_{10}^{(1)} & g_{10}^{(2)} & g_{10}^{(3)} & g_{11}^{(1)} & g_{11}^{(2)} & g_{11}^{(3)} & \dots & g_{1m}^{(1)} & g_{1m}^{(2)} & g_{1m}^{(3)} \\ g_{20}^{(1)} & g_{20}^{(2)} & g_{20}^{(3)} & g_{21}^{(1)} & g_{21}^{(2)} & g_{21}^{(3)} & \dots & g_{2m}^{(1)} & g_{2m}^{(2)} & g_{2m}^{(3)} \\ g_{30}^{(1)} & g_{30}^{(2)} & g_{30}^{(3)} & g_{31}^{(1)} & g_{31}^{(2)} & g_{31}^{(3)} & \dots & g_{3m}^{(1)} & g_{3m}^{(2)} & g_{3m}^{(3)} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \end{bmatrix}$$

举例: 图6.2的(3,2,1)的生成矩阵为

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ & 1 & 0 & 1 & 1 & 1 \\ & & 0 & 1 & 1 & 0 & 0 \\ & & & 1 & 0 & 1 & 1 & 1 & 1 \\ & & & & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Page 138



计算题举例：(3,2,m)卷积码的码字

以 u 为输入、 G 为生成矩阵的(3,2,m)的卷积码的码字为
 $V=uG$ 。

举例：图6.2的(3,2,1)卷积码，若 $u^{(1)}=(101)$ ， $u^{(2)}=(110)$ ，即
 $u=(11, 01, 10)$ ，则码字为 $V=uG=(110, 000, 001, 111)$

$$V = uG = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ & 1 & 0 & 1 & 1 & 1 & 1 \\ & 0 & 1 & 1 & 1 & 0 & 0 \\ & & 1 & 0 & 1 & 1 & 1 & 1 \\ & & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix} = [110, 110] = (110, 000, 001, 111)$$

Page 139



(n, k, m)卷积码的生成矩阵

$$G = \begin{bmatrix} G_0 & G_1 & G_2 & \dots & G_m \\ G_0 & G_1 & G_2 & \dots & G_{m-1} & G_m \\ G_0 & \dots & G_{m-2} & G_{m-1} & G_m & \dots \end{bmatrix}$$

其中每个 G_L 都是一个 $k \times n$ 阶子阵

$$G_L = \begin{bmatrix} g_{L0}^{(1)} & g_{L0}^{(2)} & \dots & g_{L0}^{(n)} \\ g_{L1}^{(1)} & g_{L1}^{(2)} & \dots & g_{L1}^{(n)} \\ \vdots & \vdots & \ddots & \vdots \\ g_{Ln}^{(1)} & g_{Ln}^{(2)} & \dots & g_{Ln}^{(n)} \end{bmatrix}$$

Page 140



卷积码编码器的存储级数

对于 (n, k, m) 卷积码，图6.1-6.2这两个例子表明，当
 $k > 1$ 时，编码器及用来描述它的符号都比较复杂，并且该
 编码器所含的 k 个移位寄存器的长度未必相同。如在图6.2
 中， $m=1$ 。

存储级数：若 k_i 是第 i 个移位寄存器的长度，则称所有 k 个
 移位寄存器中的最大长度为存储级数，即编码器的存储级
 数 m 定义为 $m = \max_{1 \leq i \leq k} k_i$

Page 141



卷积码编码器的约束长度和码速率

对于 (n, k, m) 卷积码，编码器中每个信息位要保持 $m+1$
 时间个单位，每个时间单位都可以影响编码器输出中的任
 何一个，这由移位寄存器的连接决定。

约束长度：对于 (n, k, m) 卷积码，编码器的约束长度定义为
 $n_A = n(m+1)$

约束长度可以解释成1比特信息对编码器输出可以造
 成影响的最大数目。

举例：图6.1中(2,1,2)卷积码 $n_A=6$ ，图6.2中(3,2,1)卷积码
 $n_A=6$ 。

码速率：对于 (n, k, m) 卷积码，其码速率 $r=k/n$

Page 142



(2, 1, m)卷积码的多项式描述

信息序列： u 表示成 $u(x)$ ，**生成多项式：** $g^{(1)}$ 表示成 $g^{(1)}(x)$ ， $g^{(2)}$ 表示
 成 $g^{(2)}(x)$ ，**输出序列：** $V^{(1)}$ 表示成 $V^{(1)}(x)$ ， $V^{(2)}$ 表示成 $V^{(2)}(x)$ ，

码字： V 表示成 $V(x)$ 。于是 $V^{(1)}(x)=u(x)g^{(1)}(x)$ ， $V^{(2)}(x)=u(x)g^{(2)}(x)$ ，

码字： $V(x)=V^{(1)}(x^2)+xV^{(2)}(x^2)$

举例：对于图6.1的(2,1,2)卷积码，**信息序列：**若 $u=(1011)$ ，

则 $u(x)=1+x^2+x^3$ ，**生成多项式：**冲激响应为 $g^{(1)}=(111)$ ，

$g^{(2)}=(101)$ ，则 $g^{(1)}(x)=1+x+x^2$ ， $g^{(2)}(x)=1+x^2$ ，于是，**编码方程为**

$V^{(1)}(x)=u(x)g^{(1)}(x)=(1+x^2+x^3)(1+x+x^2)=1+x+x^5$ ，

$V^{(2)}(x)=u(x)g^{(2)}(x)=(1+x^2+x^3)(1+x^2)=1+x^4+x^5$

码字为 $V(x)=V^{(1)}(x^2)+xV^{(2)}(x^2)=1+x+x^2+x^7+x^9+x^{10}+x^{11}$

Page 143



(n, k, m)卷积码的转移函数矩阵

由于编码器是线性系统，同任何有 k 个输入 n 个输出的线性系
 统一样，总共有 $k \times n$ 个转移函数。 $k \times n$ 阶转移函数矩阵为

$$G(x) = \begin{bmatrix} g_1^{(1)}(x) & g_1^{(2)}(x) & \dots & g_1^{(n)}(x) \\ g_2^{(1)}(x) & g_2^{(2)}(x) & \dots & g_2^{(n)}(x) \\ \vdots & \vdots & \ddots & \vdots \\ g_k^{(1)}(x) & g_k^{(2)}(x) & \dots & g_k^{(n)}(x) \end{bmatrix}$$

于是，码字 $V(x)=u(x)G(x)$ ，其中， $u(x)=(u^{(1)}(x), u^{(2)}(x), \dots, u^{(k)}(x))$ ， $V(x)=(V^{(1)}(x), V^{(2)}(x), \dots, V^{(n)}(x))$ ，并路之后，码字变成
 $V(x)=V^{(1)}(x^m)+xV^{(2)}(x^m)+\dots+x^{m-1}V^{(n)}(x^m)$

Page 144

图6.2的(3,2,1)卷积码, 2×3 阶转移函数矩阵为

$$G(x) = \begin{bmatrix} 1+x & x & 1+x \\ x & 1 & 1 \end{bmatrix}$$

对于输入序列 $u^{(1)}(x)=1+x^2$, $u^{(2)}(x)=1+x$

于是, 编码方程

$$\begin{aligned} V(x) &= (v^{(1)}(x), v^{(2)}(x), v^{(3)}(x)) = (1+x^2, 1+x) \begin{bmatrix} 1+x & x & 1+x \\ x & 1 & 1 \end{bmatrix} \\ &= (1+x^3, 1+x^3, x^2+x^3) \end{aligned}$$

并路之后, 码字变成 $V(x)=V^{(1)}(x^2)+xV^{(2)}(x^2)+x^2V^{(3)}(x^2)=$

$$1+x+x^8+x^9+x^{10}+x^{11}$$

Page 143

在图6.3中, 向上的实线代表输入比特为0, 向下的虚线代表输入比特为1, 对应的码字输出表示在树图的分支路径上。如输入为(1011)则输出的码字为(11, 10, 00, 01)。

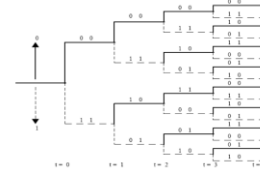


图6.3 图6.1卷积码的电路输入4个比特时对应的码树

Page 144

在图6.4中, 圆圈里是卷积码电路移位寄存器的状态, 每个输入比特都会导致卷积码电路图从一个状态到另外一个状态的转变; 每条路径边上标着 k/n , 表示输入信息比特 k 和输入码字 n 。默认从全0状态开始。如输入为(1011), 则状态转换序列为(10,01,10,11), 输出的码字为(11,10,00,01)。

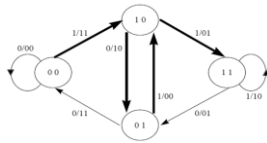


图6.4 图6.1卷积码的电路图对应的状态图

Page 147

Trellis图经常伴随一个图例(Legend), 给出状态的转换以及对应的输入输出 k/n 等。

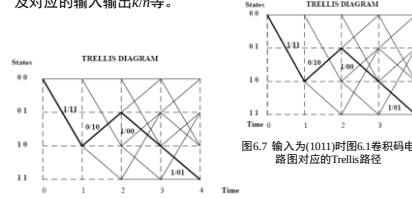


图6.6 图6.1卷积码的电路图对应的Trellis图和图例

Page 148

(n,k,m) 卷积码, 如输入序列为 x , 输出码字为 c , 经过有噪声的信号传输后, 得到接收序列 r , 然后经过Viterbi译码器, 得到序列 y 。

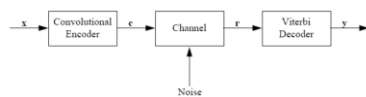


图6.8 卷积码编码及Viterbi译码系统

Page 149

Viterbi算法计算从接收序列到估值序列的最大似然估计使得 $p(r|y)$ 这个条件概率最大

对于码率为 r 的 (n,k,m) 卷积码, 输入的信息长度为 L , 则输入序列为

$$x = (x_0^{(1)}, x_0^{(2)}, \dots, x_0^{(k)}, x_1^{(1)}, x_1^{(2)}, \dots, x_1^{(k)}, \dots, x_{L+m-1}^{(1)}, x_{L+m-1}^{(2)}, \dots, x_{L+m-1}^{(k)})$$

对应输出的码字序列为

$$c = (c_0^{(1)}, c_0^{(2)}, \dots, c_0^{(n)}, c_1^{(1)}, c_1^{(2)}, \dots, c_1^{(n)}, \dots, c_{L+m-1}^{(1)}, c_{L+m-1}^{(2)}, \dots, c_{L+m-1}^{(n)})$$

其中下标表示时间。

消息序列的尾部需要 m 个0使得移位寄存器的状态归0。一般的, 默认移位寄存器从全零状态开始, 最终还是以全零状态结束。

Page 150

Viterbi译码算法是目前最大似然译码最知名一种方法，其原则为：

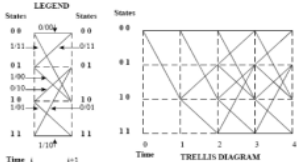
- (1) 错误不是经常出现的，发生错误的概率很小；
- (2) 在一行中发生2个错误的概率要比发生一个错误的概率小很多，即错误是随机分布的。

在工作过程中，Viterbi译码器计算给定长度的整个接收序列的度量。Viterbi译码器计算所有路径的度量并且基于这些度量进行决策。所有的路径都会进行度量累积，直到有2个路径收敛到同一个点上。这时，具有高度量的路径就是幸存者路径，否则就被丢弃。

最常用的度量就是汉明距离。

随着译码在Trellis图上按时间进行推进，路径上的度量累积起来，到最后，具有最大度量的那条路径就是幸存者路径。

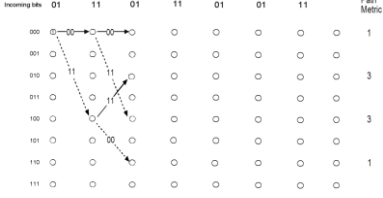
例如：假设接收序列为(01, 11 01, 11, 01, 01, 11)，在Trellis图上用Viterbi译码算法进行译码。



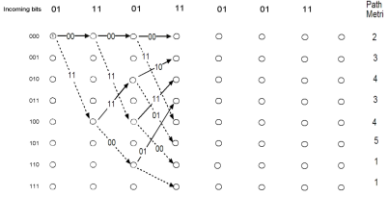
步骤1: $t=0$ 时，接收向量是01，译码器最初状态是000。从这个点出发有2条备选路径，但是哪条路径上的向量跟接收向量都不匹配。译码器计算这2条路径度量，继续同时沿着两条路径往下进行，这2条路径度量都是1。



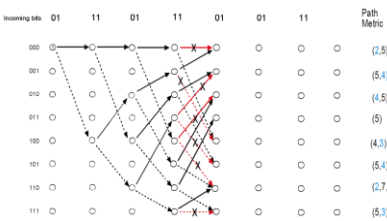
步骤2: $t=1$ 时，接收向量是11，译码器从这两个可能的状态出发，就有4条不同的路径，译码器计算这4条路径度量，并把这一些累加的尺度值标在图的右侧。



步骤3: $t=2$ 时，接收向量是01，译码器从这4个可能的状态出发，就有8条不同的路径，译码器计算这8条路径度量，把累加的尺度值标在右侧。



步骤4: $t=3$ 时，每个点都至少有一条到达它的路径。



步骤5: $t=4$ 时, 根据最大似然准则, 将每行度量大的路径留下来。

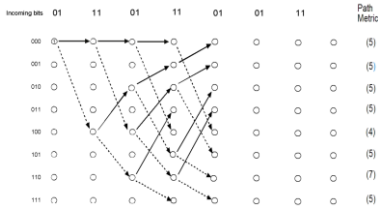


图6.13 Viterbi译码算法步骤($t=4$)

步骤6: $t=5$ 时, 继续把累加的尺度值标在右侧, 并且把每行度量大的路径留下来。

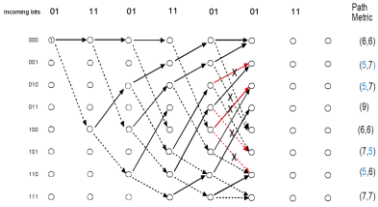


图6.14 Viterbi译码算法步骤($t=5$)

步骤7、步骤8都类似操作, 最终我们得到一条幸存者路径, 这条路径分别经过这些状态000, 100, 010, 101, 110, 011, 001, 000, 对应的译码后的码字也可以对应得到。

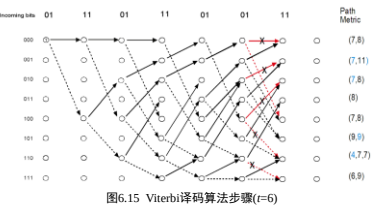


图6.15 Viterbi译码算法步骤($t=6$)

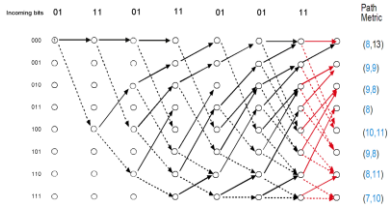


图6.16 Viterbi译码算法步骤($t=7$)

序列译码算法的描述最好是用类比。假设到一个地方去有多条路, 有的路上是有一些路标, 有些地方没有路标或者有路标但无法进行识别, 我们也许会走错路, 当我们意识到走错路的时候, 我们会回退到一个有路标的地点, 然后选择另外一条路径走下去, 如此往复, 直到我们到达目的地。在这个过程中, 我们有可能会回退多次。

在序列译码算法中, 类似的, 在Trellis图上, 每次我们只能沿着一条路径进行译码, 序列译码既允许往前走也允许往后退, 译码器记录我们的选择, 一旦走到两难的路上, 就会做标记, 当标记的数值不断增加, 超过某个预先设定的门限值的时候, 就会回退。

表6.1 (2,1,4)卷积码的真值表

Input Bits	Input State				Output Bits				Output State			
I_1	I_2	S_1	S_2	S_3	O_1	O_2	S_1	S_2	S_3	S_4	S_5	S_6
0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	1	1	1	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0	0	0	0	0	0
1	0	1	0	1	1	1	0	0	0	0	0	0
0	0	1	1	0	0	0	1	0	0	0	0	0
1	0	1	1	1	1	1	0	0	0	0	0	0
0	1	0	0	0	0	0	1	0	0	0	0	0
1	1	0	0	1	1	1	0	0	0	0	0	0
0	1	0	1	0	0	1	0	0	0	0	0	0
1	1	0	1	1	1	1	0	0	0	0	0	0
0	1	1	0	0	0	0	1	0	0	0	0	0
1	1	1	0	1	1	1	0	0	0	0	0	0
0	1	1	1	0	0	1	1	0	0	0	0	0
1	1	1	1	1	1	1	1	0	0	0	0	0

假设发送比特序列(1011000), 最后3比特是尾巴比特, 是为了使得移位寄存器可以归零。

如果没有任何错误发生, 将会接收到(11,11,01, 11,01,01,11), 但是现在实际接收到的是(01,11, 01,11,01,01,11), 发生了一个错误。

假设回退的门限为3, 下面我们序列译码算法进行译码如下。

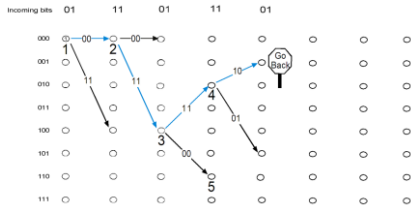


图6.17 序列译码算法

Page 143

点1: $t=0$ 时, 接收向量是01, 在点1, 译码器看两个路径, 哪个路径上的向量和接收向量01都不匹配, 并且走哪条路径, 都是发生了一个错误。假设译码器随机选择上面的分支路径00, 继续往下进行, 此时译码器把输入比特译成0, 把1放在误差计数器里。

点2: 接收向量是11, 所以直接选择下面的那条路径往下继续, 这一步没有误差。

点3: 接收向量是01, 继续走哪条路径, 都是发生了一个错误, 假设译码器随机选择上面的分支路径11, 继续往下进行, 把1放在误差计数器里, 误差计数器里的数值增加1。

点4: 接收向量是11, 继续走哪条路径, 都是发生了一个错误, 假设译码器随机选择上面的分支路径10, 然后误差计数器里到达门限值3, 所以必须回退到点4, 选择下面的分支路径01, 然后误差计数器里到达门限值3, 所以就回退到点4的上一级点3。

Page 144

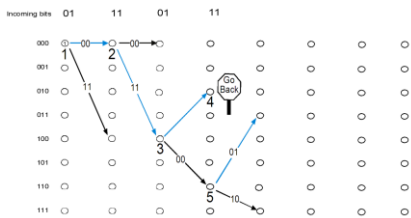


图6.18 序列译码算法

Page 145

点3: 选择下面那条路径00继续, 误差计数器里的数值增加1, 到达点5

点5: 跟刚才到达点4的情况类似, 无论选择哪条分支路径, 误差计数器里到达门限值3, 所以就回退到点3的上一级点2继续

点2: 选择上面的分支路径00, 此时误差计数器里到达门限值3, 所以回退到点1继续。

这样类似操作下去, 最终我们得到一条路径。

Page 146

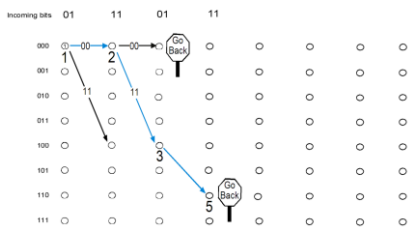


图6.19 序列译码算法

Page 147

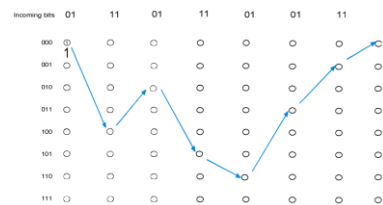


图6.20 序列译码算法

Page 148